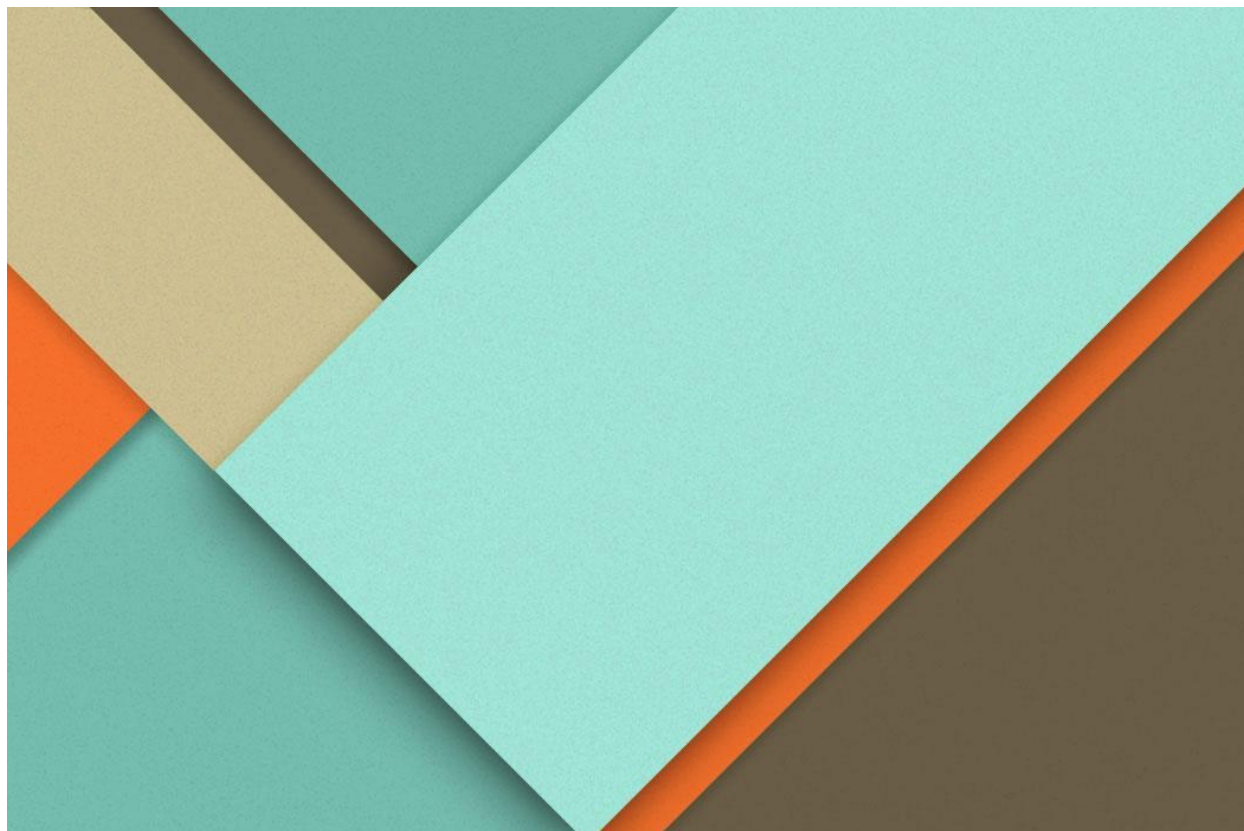




پروژه چهارم شبکه

کسرا نوربخش ۸۱۰۱۰۰۲۳۰

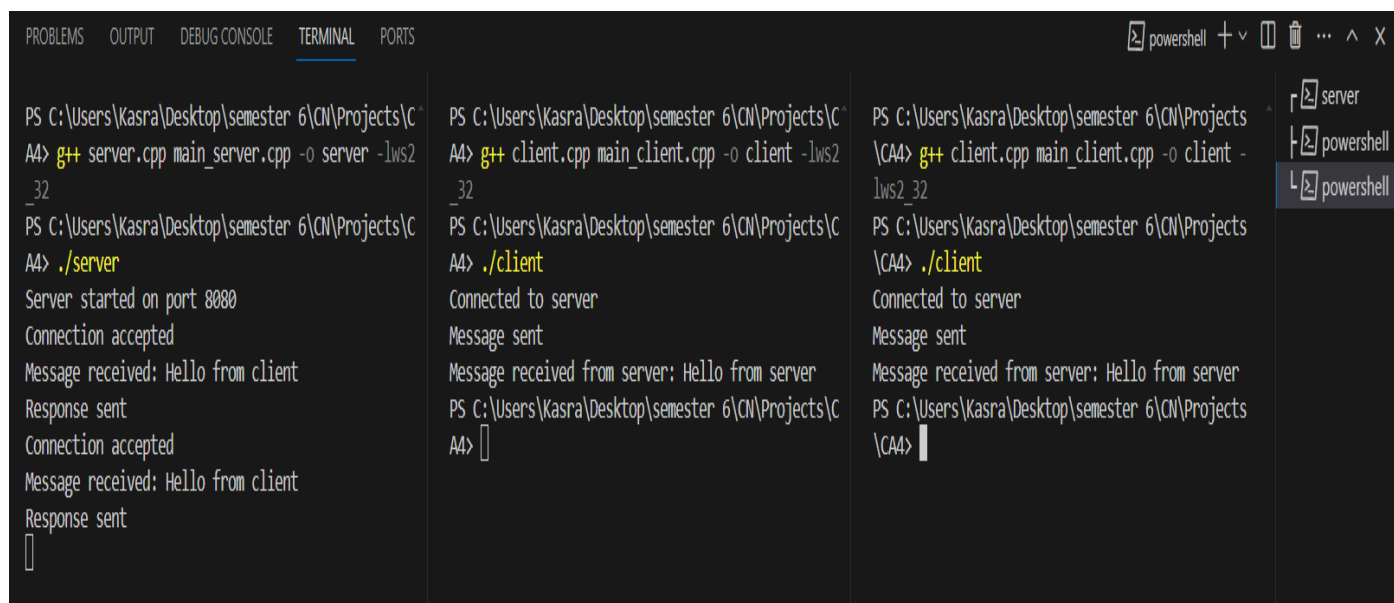
سید مهدی سید حاجی حسین ۸۱۰۱۰۰۱۱۸

مقدمه

در این پروژه برخی از مفاهیم TCP بررسی شدند که از نرم افزار `vscode` و زبان `c++` استفاده کردیم. در این پروژه هر کدام از بخش ها را در یک دیوایس و به کمک همدیگر انجام داده ایم.

گام اول

در این گام ابتدا یک سرور و کلاینت ساده درست کردیم که به همدیگر وصل شوند و پیامی رد و بدل کنند و مالتی تردینگ را هم استفاده کردیم که نتیجه را در عکس زیر می بینید:



```
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4> g++ server.cpp main_server.cpp -o server -lws2_32
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4> ./server
Server started on port 8080
Connection accepted
Message received: Hello from client
Response sent
Connection accepted
Message received: Hello from client
Response sent

PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4> g++ client.cpp main_client.cpp -o client -lws2_32
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4> ./client
Connected to server
Message sent
Message received from server: Hello from server
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4>

PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4> g++ client.cpp main_client.cpp -o client -lws2_32
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4> ./client
Connected to server
Message sent
Message received from server: Hello from server
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C\A4>
```

عکس اول: ارتباط 2 کلاینت به سرور و پیام رد و بدل شده

در ادامه هم شبیه سازی 3way-handshake را انجام دادیم با استفاده از متد `performHandshake()` و سپس انتقال داده با استفاده از متد های `receiveData(SOCKET client_socket)` و `receiveAck()`

که نتیجه را در عکس های زیر میبینیم:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
AA> ./server
Server started on port 8080
Connection accepted
SYN received
SYN-ACK sent
ACK received, handshake completed
Message received: Hello from client
Response sent
Connection accepted
SYN received
SYN-ACK sent
ACK received, handshake completed
Message received: Hello from client
Response sent

PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C...
AA> ./client
Connected to server
SYN sent
SYN-ACK received
ACK sent, handshake completed
Message sent
Message received from server: Hello from server
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\C
AA>

PS C:\Users\Kasra\Desktop\semester 6\CN\Projects
\CA4> ./client
Connected to server
SYN sent
SYN-ACK received
ACK sent, handshake completed
Message sent
Message received from server: Hello from server
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects
\CA4>
```

عکس دوم: پیام های SYN, SYN-ACK, ACK

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\CA4> ./server
Server started on port 8080
Connection accepted
SYN received
SYN-ACK sent
ACK received, handshake completed
Data received: Hello from client
ACK sent
Data received: This is a PACKET
ACK sent

PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\CA4> ./client
Connected to server
SYN sent
SYN-ACK received
ACK sent, handshake completed
Message sent: Hello from client
ACK received from server
Message sent: This is a PACKET
ACK received from server
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\CA4>
```

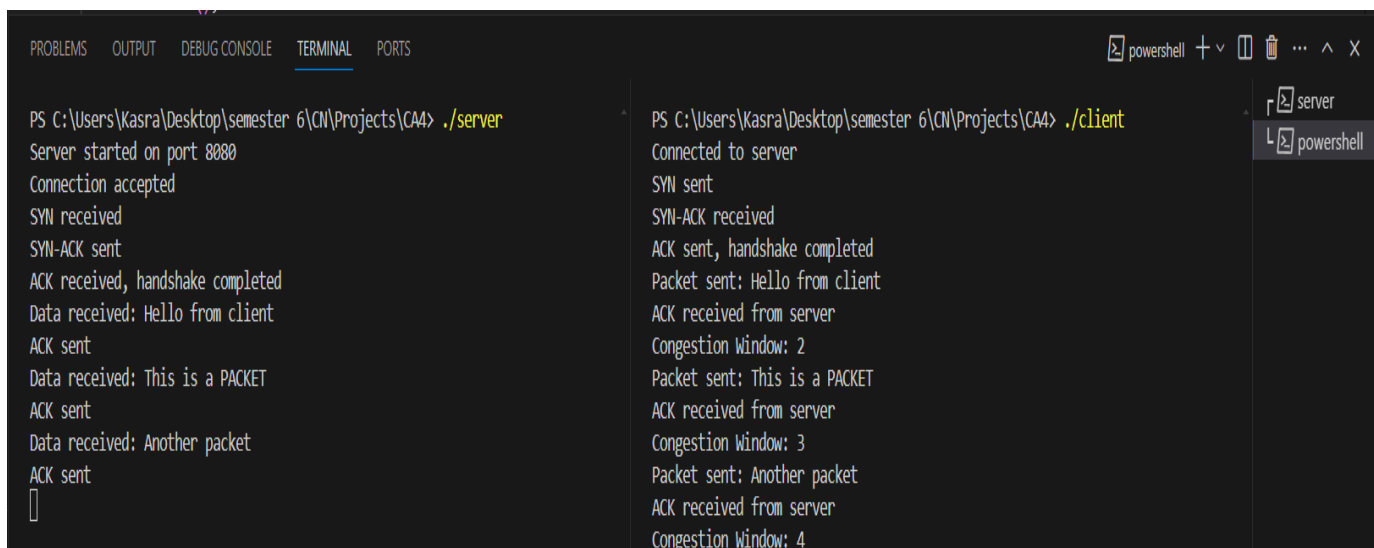
عکس سوم: انتقال 2 بسته داده و گرفتن ACK آن ها

گام دوم

ابتدا به توضیح این الگوریتم می پردازیم. الگوریتم کنترل ازدحام TCP New Reno به منظور بهبود کارایی انتقال داده ها در شبکه طراحی شده است. این الگوریتم با فاز "شروع آهسته" آغاز می شود، که در آن اندازه

پنجره تراکم (cwnd) به صورت نمایی افزایش می‌یابد. پس از رسیدن به یک آستانه مشخص، فاز "اجتناب از ازدحام" شروع می‌شود و اندازه پنجره تراکم به صورت خطی افزایش می‌یابد. در صورت دریافت سه ACK تکراری که نشان‌دهنده گم شدن یک بسته است، بسته گمشده بلافاصله بازارسال می‌شود و الگوریتم به جای بازگشت به حالت شروع آهسته، به حالت "بازیابی سریع" می‌رود. در این حالت، اندازه پنجره تراکم به مقدار آستانه تنظیم شده و تا دریافت ACK برای بسته گمشده، به صورت کنترل‌شده افزایش می‌یابد. این روش به TCP New Reno اجازه می‌دهد تا بهبود قابل توجهی در بازدهی انتقال داده‌ها و مدیریت مؤثرتر تراکم در شبکه‌های پیچیده داشته باشد.

حال بعد از پیاده سازی جز به جز الگوریتم گفته شده نتیجه را میبینیم:



```
PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\CA4> ./server
Server started on port 8080
Connection accepted
SYN received
SYN-ACK sent
ACK received, handshake completed
Data received: Hello from client
ACK sent
Data received: This is a PACKET
ACK sent
Data received: Another packet
ACK sent
[]

PS C:\Users\Kasra\Desktop\semester 6\CN\Projects\CA4> ./client
Connected to server
SYN sent
SYN-ACK received
ACK sent, handshake completed
Packet sent: Hello from client
ACK received from server
Congestion Window: 2
Packet sent: This is a PACKET
ACK received from server
Congestion Window: 3
Packet sent: Another packet
ACK received from server
Congestion Window: 4
```

گام سوم

در این بخش با استفاده از یک خیلی ساده یک فایل 2 مگابایتی پر شده از A با اسم "input.txt" ساخته شد تا ارسال شود و در فایل "output.txt" ذخیره شود که میبینیم برنامه درست کار کرده است و زمان ارسال هم است.

سوالات

سوال اول: پروتکل Go-Back-N (GBN) نوعی پروتکل sliding window است که برای انتقال قابل

اعتماد داده ها در شبکه استفاده می شود. این پروتکل بخشی از خانواده پروتکل های Automatic

Repeat request (ARQ) است که برای اطمینان از انتقال دقیق داده ها طراحی شده اند. ارسال بسته ها

به این صورت است که فرستنده تا اندازه پنجره N بسته ها را بدون انتظار برای ACK ارسال می کند. در هنگام

دریافت، گیرنده شماره توالی هر بسته ورودی را بررسی می کند، اگر بسته همان بسته مورد انتظار باشد، آن را

ACK می کند و به بسته بعدی مورد انتظار می رود. اگر بسته خارج از ترتیب باشد، آن را نادیده می گیرد و ACK

برای آن ارسال نمی کند. اگر تایمر فرستنده برای یک بسته خاص منقضی شود و نشان دهد که تأییدیه ای در

مدت زمان معین دریافت نشده است، فرستنده آن بسته و تمام بسته های بعدی در پنجره را مجدداً ارسال

می کند. از مزایای آن میتوان به: سادگی و استفاده مؤثر از پهنای باند وقتی نرخ خطا کم است اشاره کرد. از

معایب هم: گیرنده بسته های خارج از ترتیب را بافر نمی کند، که منجر به هدر رفتن منابع شبکه می شود.

سوال دوم: پروتکل Selective Repeat یکی دیگر از پروتکل های sliding window و بخشی از

خانواده پروتکل های Automatic Repeat request (ARQ) است. این پروتکل برای بهبود کارایی

انتقال داده ها در مقایسه با پروتکل Go-Back-N طراحی شده است. همانند GBN فرستنده و گیرنده هر دو

از پنجره ای با اندازه N استفاده می کنند اما به گیرنده اجازه می دهد تا بسته هایی را که خارج از ترتیب دریافت

کرده، ذخیره کند تا زمانی که بسته های گم شده یا خراب به درستی دریافت شوند. اگر یک بسته گم شود یا

خراب شود، گیرنده فقط درخواست ارسال مجدد آن بسته خاص را می کند و سایر بسته های دریافت شده را نگه

می دارد. مزیت آن به طور کلی این است که فرستنده تنها بسته هایی را که تأییدیه ای برای آن ها دریافت نکرده

یا درخواست ارسال مجدد شده اند، ارسال می کند. این پروتکل به ویژه در شرایطی که نرخ خطا بالا است، به

دلیل بافر کردن بسته های خارج از ترتیب و ارسال مجدد تنها بسته های گم شده یا خراب، بسیار مؤثر است.